Guía Arquitectónica y Tutorial: Desarrollo de un Agente Asistente Personal Autónomo con OpenClaw y Notion

El desarrollo de la inteligencia artificial agentiva representa una transición arquitectónica fundamental en la ingeniería de software, marcando la evolución desde modelos generativos reactivos y sin estado hacia sistemas dirigidos por objetivos que son capaces de percibir, planificar, actuar y adaptarse a través de bucles de control iterativos.[1] En el ámbito de la productividad personal y empresarial, la implementación de un agente autónomo que gestione calendarios, clasifique tareas, estructure itinerarios complejos y programe recordatorios dinámicos exige una orquestación sumamente precisa entre el Modelo de Lenguaje Grande (LLM), el marco de ejecución subyacente y las interfaces de programación de aplicaciones (API) externas.[2]

La restricción fundamental de mantener un entorno de alta seguridad y privacidad, excluyendo explícitamente el ecosistema de Google y sus derivados por políticas de protección de datos, obliga a diseñar una arquitectura que dependa de proveedores alternativos robustos y de una configuración minuciosa de los permisos de acceso.[2] Asumiendo que el marco de ejecución OpenClaw ya se encuentra configurado e integrado en el entorno de despliegue como una puerta de enlace persistente (gateway) [3], el presente documento técnico se articula como un reporte y tutorial exhaustivo paso a paso. Se detallan las decisiones arquitectónicas para la selección del LLM, la evaluación de proveedores de calendario alternativos, la esquematización de bases de datos para la clasificación de tareas, la configuración de motores de recordatorios proactivos, la ingeniería de memoria cognitiva y las optimizaciones críticas de rendimiento necesarias para viabilizar económicamente un asistente personal de nivel empresarial operando de manera ininterrumpida.[1]

Elección del Motor de Razonamiento (LLM) y Configuración de Claves de API

El núcleo cognitivo del agente asistente no debe evaluarse únicamente por su fluidez en la generación de lenguaje natural, sino por su competencia funcional estricta: la capacidad de llamar a herramientas externas (tool calling) en la secuencia correcta, emitir salidas estructuradas invariables que los sistemas informáticos puedan interpretar, y recuperarse autónomamente de errores de ejecución en la red.[5] La selección del LLM adecuado impacta directamente en la latencia operativa, el costo financiero sostenido y la fiabilidad matemática de la programación de tareas temporales.

El ecosistema de modelos en 2025 y 2026 presenta variaciones significativas en costos y capacidades, lo que requiere un análisis comparativo para determinar el motor óptimo para la toma de decisiones del agente.

Modelo de Lenguaje	Costo de Entrada (por 1M tokens)	Costo de Salida (por 1M tokens)	Capacidad de Invocación de Herramientas (Tool Calling)	Casos de Uso Recomendados en el Agente
Claude 3.5 / 3.7 Sonnet	$3.00	$15.00	Excepcional	Planificación central, resolución de conflictos de agenda, reestructuración semántica de itinerarios.[7]
GPT-4o	$2.50	$10.00	Muy Alta	Procesamiento general, alta velocidad de respuesta y adherencia estricta a esquemas de salida.[7]
Gemini 1.5 Pro	$1.25	$5.00	Alta	Análisis de ventanas de contexto masivas, útil para procesar historiales anuales completos de tareas.[7]
Llama 3.3 70B	$0.50 - $0.90	$0.50 - $0.90	Moderada / Alta	Tareas de enrutamiento básico o recordatorios simples mediante proveedores de inferencia rápida.[10]
DeepSeek V3 / R1	< $0.20	< $0.50	Alta	Reducción extrema de costos en tareas de razonamiento iterativo continuo o análisis de fondo.[7]

GPT-4o mini	$0.15	$0.60	Moderada	Tareas de alto volumen y baja complejidad, como verificación de estado o resúmenes diarios.[8]

Para configurar el agente correctamente, el primer paso tutorial requiere inyectar la clave de la API (API Key) del proveedor seleccionado en el entorno de OpenClaw. La arquitectura de OpenClaw exige que las credenciales sensibles nunca se incluyan directamente en las instrucciones de texto, sino que se alojen de manera segura en el archivo ~/.openclaw/.env o se configuren mediante la interfaz de línea de comandos, permitiendo que el marco inyecte estas variables de entorno de forma dinámica al inicializar el contenedor o entorno de ejecución del agente.[12] Esta práctica garantiza que si la base de datos de conocimiento del agente es respaldada en repositorios externos, las claves criptográficas permanezcan seguras en el host local.

La elección del LLM principal para el planificador de calendarios recae sobre Claude 3.5 Sonnet o GPT-4o, dada su supremacía en la adherencia a esquemas JSON. Al interactuar con APIs externas, el agente debe emitir cargas útiles que coincidan exactamente con las expectativas del servidor. Mientras que proveedores como OpenAI ofrecen funcionalidades de salidas estructuradas nativas, arquitecturas basadas en Anthropic implementan esquemas de herramientas donde se fuerza el uso de un objeto JSON Schema estricto validado del lado del cliente.[14] Si se optara por un modelo local de bajo costo por razones de privacidad extrema, como Mistral o modelos de la serie GLM, la salida JSON puede ser inconsistente, requiriendo que la arquitectura del agente incorpore herramientas de validación intermedia, tales como analizadores Pydantic, para reparar cadenas malformadas antes de enviarlas a la API del calendario.[14]

Evaluación y Elección del Proveedor de Calendario

La exclusión explícita de los servicios de Google por imperativos de seguridad reduce drásticamente las opciones de integración estándar, obligando a evaluar plataformas que ofrezcan control soberano de los datos, capacidades programáticas avanzadas y soporte para metadatos complejos que permitan la clasificación de tareas.[2]

La primera alternativa evaluada es Proton Calendar, reconocido mundialmente por su implementación estricta de cifrado de extremo a extremo. Aunque representa el estándar más alto en privacidad documental y de comunicaciones, Proton carece de una API pública funcional y documentada para desarrolladores a fecha de 2025 y principios de 2026. Sus capacidades de interacción programática están severamente restringidas o delegadas a hojas de ruta futuras, lo que imposibilita de facto la integración de un agente autónomo que requiera crear, modificar o consultar eventos dinámicamente mediante llamadas HTTP autorizadas.[17]

La segunda alternativa se basa en infraestructuras autohospedadas como Nextcloud, la cual opera utilizando el protocolo estándar de la industria CalDAV (descrito en el RFC 4791). Este

protocolo soporta de forma nativa componentes de eventos temporales (VEVENT) y tareas pendientes (VTODO), permitiendo que un agente interactúe con el servidor a través de bibliotecas de Python como caldav para gestionar agendas y sincronizar operaciones.[20] Sin embargo, el principal obstáculo técnico de CalDAV radica en su rigidez estructural. Para que el agente pueda clasificar tareas según propiedades multidimensionales complejas (como nivel de energía requerido, proyecto asociado o urgencia), se debe forzar la inyección de atributos personalizados no estándar, conocidos como X-PROPS o X-CALDAVSYNCHRONIZER. Esta manipulación incrementa exponencialmente la fragilidad de la integración, complicando el análisis (parsing) del agente y generando inconsistencias cuando los datos se sincronizan con clientes móviles convencionales.[22]

Ante estas limitaciones, Notion emerge como la solución arquitectónica superior. Aunque no es un servidor CalDAV en el sentido tradicional, Notion opera como un motor de base de datos relacional híbrido que integra vistas de calendario, tableros Kanban y gestión de documentos de forma nativa.[25] La API de Notion permite a los desarrolladores definir esquemas de datos altamente granulares, ideales para agentes de inteligencia artificial que requieren modelar y clasificar tareas basándose en lógicas condicionales profundas.[27] A partir de la actualización del modelo de datos de la API de Notion en septiembre de 2025, existe una separación lógica clara entre el objeto de base de datos y sus fuentes de datos (data sources), lo que permite al agente interactuar con los esquemas de manera más eficiente y consultar colecciones de páginas estructuradas sin ambigüedad.[30]

La decisión de utilizar Notion proporciona al agente un entorno unificado donde los eventos temporales no son meros bloques en un calendario, sino entidades de datos ricas que pueden vincularse a bases de conocimiento adyacentes, documentos de proyectos e historiales de progreso.[25]

Diseño del Esquema de Base de Datos en Notion para Clasificación de Tareas

Para que el agente asistente pueda asumir el rol de planificador, clasificador y generador de itinerarios, es un prerrequisito técnico establecer un esquema de base de datos en Notion que traduzca el lenguaje natural del usuario a estructuras de datos rígidas. El segundo paso del tutorial consiste en crear esta base de datos matriz en el espacio de trabajo de Notion y exponerla al agente.[29]

El agente interactuará con las siguientes propiedades fundamentales para inyectar, actualizar y recuperar información del calendario y de las listas de tareas pendientes [27]:

Nombre Lógico de la Propiedad	**Tipo de Dato en Notion API**	**Propósito Arquitectónico para el Agente Autónomo**
Nombre de la Tarea	title	Constituye el identificador principal del evento o tarea, operando como la clave primaria textual de la página

		generada.[27]
Programación Temporal	date	Rango temporal formateado en estándar ISO 8601, incluyendo hora y zona horaria. Admite parámetros de inicio (start) y fin (end) para la representación en el calendario.[27]
Estado de Ejecución	status	Define la progresión del flujo de trabajo (por ejemplo, *To-Do*, *In Progress*, *Done*). Es vital para que el agente construya consultas que filtren tareas ya completadas.[28]
Contexto Operativo	multi_select	Clasificación del entorno de ejecución de la tarea, permitiendo etiquetas múltiples como *Trabajo Profundo*, *Reunión*, *Administrativo*, o *Personal*.[28]
Nivel de Prioridad	select	Determina la urgencia paramétrica de la tarea, permitiendo al agente ordenar itinerarios en función de clasificaciones como *Baja*, *Media*, *Alta* o *Crítica*.[28]
Requiere Acción de IA	checkbox	Bandera booleana que actúa como un disparador (trigger) silencioso. Permite que rutinas en segundo plano identifiquen si el agente debe procesar esta tarea sin intervención humana adicional.[28]

Una vez definida la estructura en la interfaz gráfica de Notion, el administrador debe crear una Integración Interna en la consola de desarrolladores de Notion, obtener el token de portador (Bearer Token) y compartir explícitamente la base de datos recién creada con dicha integración para concederle permisos de lectura y escritura.[31] Este token se introduce posteriormente en el ecosistema OpenClaw utilizando los comandos de configuración de habilidades, asegurando que el agente adquiera autorización criptográfica para acceder al esquema.[34]

Es imperativo comprender las limitaciones técnicas de esta arquitectura. La API pública de Notion recomienda mantener el tamaño total del esquema de la base de datos por debajo de los 50 kilobytes para garantizar un rendimiento óptimo de latencia durante las consultas del agente. Además, la API actualmente no soporta la manipulación directa de bases de datos

vinculadas (Linked databases) ni la creación programática de bases de datos de tipo Wiki, lo que exige que el agente opere exclusivamente sobre fuentes de datos estándar.[26]

Construcción de la Habilidad (Skill) en OpenClaw y Lógica de Itinerarios

El marco OpenClaw abstrae la complejidad de la conexión de herramientas mediante un sistema de extensiones modulares denominado Habilidades (Skills). En lugar de requerir la compilación de código de middleware complejo, una habilidad se define como un directorio que aloja un único archivo SKILL.md.[36] Este archivo cumple una función dual: emplea metadatos YAML en su encabezado para declarar dependencias técnicas, y utiliza Markdown en su cuerpo para proporcionar el manual de instrucciones operativas ("runbook") que dictamina el comportamiento cognitivo del LLM.[36]

El tercer paso en la implementación tutorial exige la creación de este entorno. El administrador del sistema debe generar un directorio específico dentro del espacio de trabajo del agente, típicamente ubicado en ~/.openclaw/workspace/skills/gestor-calendario-notion/, y crear el archivo SKILL.md correspondiente.[36]

La estructura interna del archivo SKILL.md debe modelarse cuidadosamente para dotar al agente de lógica deductiva para la manipulación del tiempo y el espacio en la agenda del usuario.[40] El encabezado YAML declara el nombre de la habilidad, su descripción y los requisitos de entorno, verificando en tiempo de carga que el contenedor de ejecución posea acceso a la variable NOTION_API_KEY.[36] El bloque de instrucciones en Markdown detalla la lógica de planificación (ReAct). Se instruye al agente para que extraiga los parámetros temporales de solicitudes formuladas en lenguaje natural, obligándolo a convertir expresiones ambiguas como "la próxima semana" o "pasado mañana por la tarde" en cadenas de tiempo estrictas en formato ISO 8601, ajustadas a la zona horaria local del usuario antes de su serialización a UTC para la API.[2]

Para que el agente pueda generar un itinerario diario coherente, la lógica de la habilidad debe estructurarse en fases precisas. Primero, el agente requiere recopilar la información del estado actual mediante consultas a la API de Notion. La API de Notion es extremadamente flexible y permite aplicar filtros compuestos utilizando operadores lógicos and y or con hasta dos niveles de anidación.[33] Por lo tanto, en lugar de solicitar todo el contenido del calendario, el agente genera una petición JSON que filtra exclusivamente los eventos cuya fecha coincida con el día objetivo y cuyo estado no esté marcado como completado.

Al recibir la matriz de tareas, el agente evalúa los bloques de tiempo. La instrucción del SKILL.md le obliga a identificar las obligaciones con un horario de inicio y fin rígido, calculando posteriormente los espacios en blanco (whitespace) disponibles entre ellas.[2] En esos intervalos libres, el agente debe programar las tareas pendientes, agrupando aquellas que compartan la misma etiqueta de contexto (por ejemplo, agrupar todas las tareas "Administrativas" en un solo bloque matutino) para minimizar la fatiga por cambio de contexto en el usuario. Si el agente detecta conflictos o solapamientos de agenda durante este cálculo algorítmico, las directivas le exigen que seleccione el momento menos perjudicial para anular eventos de baja prioridad o

que formule una sugerencia de reprogramación al usuario antes de proceder con una confirmación destructiva.[40]

Configuración de Recordatorios: Sistemas Cron y Heartbeat

La verdadera utilidad de un asistente personal no reside meramente en su capacidad de respuesta a comandos directos, sino en su proactividad funcional. Un agente de alto nivel debe operar de forma autónoma en segundo plano, supervisando plazos y emitiendo alertas. OpenClaw resuelve este requerimiento arquitectónico proporcionando dos motores de activación programada: el programador Cron y el sistema Heartbeat.[42]

El cuarto paso del tutorial abarca la configuración de estos motores. El sistema Cron está diseñado para la ejecución de flujos de trabajo en momentos temporales exactos y predecibles. Resulta ser la herramienta ideal para compilar y enviar itinerarios matutinos diarios o resúmenes nocturnos de cierre de jornada. Para evitar que la inyección masiva de datos contamine el historial de conversación continuo que el usuario mantiene con el agente, los trabajos Cron deben ejecutarse en un entorno aislado utilizando el parámetro de sesión aislada.[43]

A través de la interfaz de línea de comandos de OpenClaw, el administrador puede programar un trabajo recurrente que despierte al agente cada mañana. La instrucción técnica define el nombre del trabajo, la expresión temporal estándar de Unix, la zona horaria de referencia, el aislamiento de la sesión, y el mensaje de sistema inyectado. Este mensaje indicará al agente que consulte la base de datos de Notion, analice las reuniones del día, ordene las tareas pendientes según la propiedad de prioridad, e informe proactivamente al usuario enviando un mensaje directo a través de un canal integrado como Slack o Telegram.[43]

Por otro lado, el sistema Heartbeat (latido) se encarga de los recordatorios dinámicos y la supervisión de condiciones cambiantes. Mientras que Cron ejecuta órdenes a horas fijas, el Heartbeat despierta al agente de forma continua en intervalos regulares, por ejemplo, cada treinta minutos.[43] Durante cada latido, el agente escanea silenciosamente el entorno en busca de tareas urgentes. Para implementar esto, el agente está programado para leer un archivo de estado local denominado HEARTBEAT.md o interrogar la API de Notion en busca de eventos cuya fecha de inicio programada esté próxima a expirar en los próximos minutos.[44]

La configuración del Heartbeat se define en los bloques de inicialización del sistema OpenClaw. Se especifica el intervalo de repetición, el destino de entrega de los mensajes (usualmente el último canal de comunicación activo del usuario) y, de manera crítica, las horas activas (activeHours). Configurar un bloque de horas activas, por ejemplo de 08:00 a 22:00, garantiza que el agente no ejecute llamadas a la API durante los períodos de descanso nocturno del usuario, lo que además de respetar la desconexión digital, actúa como una medida severa de contención de costos de inferencia.[46] En escenarios donde se han reportado errores de ejecución debido a la existencia de archivos HEARTBEAT.md vacíos, la mitigación comprobada consiste en estructurar el trabajo recurrente de modo que invoque directamente una carga útil de tipo agentTurn dentro de una sesión aislada, forzando así al agente a interactuar sin

depender de la lectura de metadatos locales corruptos.[44]

Arquitectura Cognitiva: Contexto Persistente vs. Contexto Efímero

Uno de los problemas de ingeniería más persistentes en el despliegue de agentes autónomos es el manejo de la memoria y la ventana de contexto. Los modelos de lenguaje modernos, independientemente de su tamaño, son sistemas inherentemente "amnésicos"; operan sin estado y olvidan toda la información entre llamadas a la API a menos que se les vuelva a suministrar.[47] Comprender la divergencia fundamental entre el contexto efímero y la memoria persistente es crucial para evitar que el asistente sufra alucinaciones, pierda referencias de proyectos a largo plazo o consuma presupuestos masivos recargando historiales irrelevantes.[47] Para esclarecer estos conceptos dentro de la arquitectura de agentes, la siguiente taxonomía define las fronteras de memoria:

Concepto Arquitectónico	Definición Técnica	Limitaciones y Comportamientos Clave
Ventana de Contexto	La capacidad máxima de tokens que el LLM puede procesar en una sola llamada de inferencia.	Es una restricción de hardware/software, no una estrategia de memoria. Saturarla degrada el rendimiento del modelo.[48]
Contexto Efímero	El estado temporal de razonamiento inyectado en un *prompt* que se destruye tras la interacción.	Suficiente para tareas aisladas sin estado, pero inútil para el aprendizaje a lo largo del tiempo o el trabajo de proyectos continuos.[48]
Memoria de Sesión	El historial de conversación que se mantiene de forma contigua durante un ciclo de trabajo activo.	Propenso al problema de la "Bomba de Tokens" a medida que crece. Eventualmente requiere compresión, perdiendo alta fidelidad.[48]
Memoria de Agente (Persistente)	Almacenamiento estructurado y duradero que sobrevive a los reinicios de hardware y límites de sesión.	Requiere infraestructura externa (archivos planos o bases de datos vectoriales) y mecanismos explícitos de recuperación.[48]

En ecosistemas en vivo, depender de la memoria de sesión inyectando historiales de conversación masivos genera el problema conocido como "Lost in the Middle" (Pérdida en el medio), donde el LLM ignora instrucciones cruciales ubicadas en el centro del bloque de texto debido a la complejidad cuadrática de los mecanismos de autoatención.[47] Para resolver esto, OpenClaw implementa un sistema avanzado de protección de contexto que involucra la

compactación activa. Cuando la ventana de contexto del LLM se acerca a su límite operativo, el sistema resume automáticamente el historial de mensajes antiguos. Sin embargo, para evitar la pérdida silenciosa de decisiones importantes del usuario, el marco ejecuta un "Flush Pre-Compactación".[51] Antes de que los datos sean resumidos y potencialmente destruidos, el agente evalúa autónomamente la conversación y escribe los hechos vitales en archivos de almacenamiento persistente en el disco local, salvaguardando el conocimiento crítico sin intervención del usuario.[51]

La arquitectura de persistencia se consolida mediante la separación del almacenamiento en niveles. La identidad del núcleo y las reglas inquebrantables se almacenan en archivos Markdown estáticos (MEMORY.md, PROFILE.md, HABITS.md) que el agente inyecta en su *prompt* del sistema al inicio de cada sesión.[51] Esto reemplaza la dependencia de herramientas como notas secundarias en la nube, moviendo el conocimiento directamente al sistema de archivos local versionado.[54]

Para memorias expansivas, como historiales de proyectos de meses de duración, inyectar archivos de texto completos se vuelve ineficiente. En estos casos, se integran plugins de memoria externa, tales como el módulo @mem0/openclaw-mem0 o sistemas basados en HyperStack.[55] Estos sistemas externalizan el conocimiento en bases de datos locales, almacenando la memoria como entidades atómicas y hechos versionados en lugar de párrafos crudos. Durante la ejecución, el agente utiliza una búsqueda híbrida que combina la similitud semántica (búsqueda vectorial) para comprender la intención general, con la recuperación por coincidencia exacta de texto completo para garantizar la precisión matemática al recordar puertos, identificadores de eventos o reglas de programación.[49] Al separar la memoria a largo plazo (centrada en las idiosincrasias y hechos del usuario) de la memoria a corto plazo (el contexto del flujo de trabajo actual), el agente logra una continuidad sin precedentes sin inflar innecesariamente el costo operativo por tokens.[55]

Optimizaciones Aplicables para Rendimiento y Reducción de Costes

El despliegue de agentes autónomos que se ejecutan perpetuamente en segundo plano, evaluando esquemas de bases de datos y procesando itinerarios diarios, puede incurrir en costos de inferencia API prohibitivos. Para viabilizar un sistema de grado profesional, se deben implementar estrategias estrictas de optimización que ataquen tanto la latencia del servicio como la facturación por consumo de tokens.[10]

La optimización tecnológica más transformadora disponible es el *Prompt Caching* (Almacenamiento en Caché de Contexto). La arquitectura de un agente requiere enviar repetidamente un enorme bloque de instrucciones iniciales, definiciones de esquemas de herramientas y el historial de comportamiento previo en cada interacción. Dado que este bloque de texto preliminar rara vez cambia drásticamente entre llamadas consecutivas, el *Prompt Caching* permite a los proveedores de inferencia retener este prefijo procesado en sus servidores.[59]

La implementación de esta tecnología difiere según la plataforma. En el ecosistema de OpenAI,

el proceso es en gran medida automático; el enrutador de la API detecta prefijos idénticos enviados recientemente y aplica descuentos de forma transparente, reduciendo los tiempos de procesamiento de solicitudes masivas.[60] En contraste, infraestructuras basadas en Anthropic (Claude) proporcionan un control granular, permitiendo a los ingenieros colocar puntos de interrupción explícitos mediante nodos efímeros (cache_control: {"type": "ephemeral"}) directamente en bloques de contenido de la carga útil. Esto indica al servidor exactamente hasta qué punto de la estructura JSON debe almacenar el estado de atención computado.[59]

El modelo económico que fundamenta esta optimización es contundente. Si un agente envía 3000 tokens de contexto persistente e instrucciones estáticas (prefijo) y genera 1000 tokens de diálogo variable (sufijo) a lo largo de 100 llamadas de evaluación en una ventana de cinco minutos, los costos difieren radicalmente [59]:

- **Sin optimización de caché**: $100 \times (3000 + 1000) = 400,000$ tokens facturados a precio regular.
- **Con optimización de caché (asumiendo un costo de token en caché del 10%)**: La primera llamada computa los 4000 tokens completos. Las 99 llamadas subsecuentes computan $99 \times (0.1 \times 3000 + 1000)$, lo que resulta en un equivalente a $132,700$ tokens facturados a precio regular.

Esta arquitectura garantiza una reducción del consumo bruto superior al 65%, habilitando la viabilidad financiera de operar un sistema de Heartbeat que evalúe calendarios múltiples veces por hora.[62]

Además del almacenamiento en caché, el Enrutamiento de Modelos (Model Routing) actúa como una segunda capa de defensa económica. Dado que OpenClaw es una plataforma agnóstica respecto al modelo subyacente [3], se pueden asignar diferentes motores de inferencia a diferentes clases de problemas. Las comprobaciones de rutina en segundo plano, la evaluación de un archivo de estado simple o el análisis de formato básico no requieren el poder deductivo profundo de un modelo de vanguardia. Estas tareas se enrutan a modelos veloces y altamente eficientes, como Gemini 1.5 Flash o Llama 3 8B, minimizando el gasto computacional. Por el contrario, las tareas complejas de deduplicación de eventos en Notion, la evaluación de colisiones horarias y el armado algorítmico del itinerario diario se derivan a Claude 3.5 Sonnet o GPT-4o para garantizar el éxito del formato de salida estructurada.[8]

Finalmente, la minimización de la carga útil (payload) durante las interacciones con APIs externas asegura la preservación del ancho de banda y del contexto. Cuando el agente de planificación emite una consulta a la base de datos de Notion, la descarga indiscriminada de todas las propiedades contamina la ventana de razonamiento y multiplica los costos. Para evitar esto, las directivas del agente exigen el uso del parámetro filter_properties en los *endpoints* de consulta, instruyendo al sistema a recuperar exclusivamente las columnas necesarias para la evaluación temporal (por ejemplo, el título, la fecha de inicio y el estado de compleción), omitiendo descripciones extensas, registros de auditoría o metadatos de usuario irrelevantes.[67] Al interceptar la proliferación de datos innecesarios en la fase de recuperación, el agente opera de manera ligera, económica y matemáticamente precisa en sus tareas de orquestación de vida personal y profesional.[2]

Works cited

1. From Prompt–Response to Goal-Directed Systems: The Evolution of Agentic AI Software Architecture - arXiv, accessed March 17, 2026, https://arxiv.org/html/2602.10479v1
2. How to Build Autonomous AI Agents for Scheduling and Reminders | MindStudio, accessed March 17, 2026, https://www.mindstudio.ai/blog/build-autonomous-ai-agents-scheduling-reminders/
3. OpenClaw (Formerly Clawdbot & Moltbot) Explained: A Complete Guide to the Autonomous AI Agent - Milvus, accessed March 17, 2026, https://milvus.io/blog/openclaw-formerly-clawdbot-moltbot-explained-a-complete-guide-to-the-autonomous-ai-agent.md
4. How OpenClaw Works: Understanding AI Agents Through a Real Architecture, accessed March 17, 2026, https://bibek-poudel.medium.com/how-openclaw-works-understanding-ai-agents-through-a-real-architecture-5d59cc7a4764
5. Top LLMs 2025: Best Models for Modern AI Systems Explained - Thesys, accessed March 17, 2026, https://www.thesys.dev/blogs/top-llms
6. Building an AI Calendar Agent from Scratch — No Frameworks, Just Pure Python and Gemini | by Jeeva V | Medium, accessed March 17, 2026, https://medium.com/@jeeva.v910/building-an-ai-calendar-agent-from-scratch-no-frameworks-just-pure-python-and-gemini-9e65e79341c4
7. LLM API Pricing Comparison (2025): OpenAI, Gemini, Claude | IntuitionLabs, accessed March 17, 2026, https://intuitionlabs.ai/articles/llm-api-pricing-comparison-2025
8. LLM API Cost Comparison: GPT-4 vs Claude vs Llama (2026) - Inventive HQ, accessed March 17, 2026, https://inventivehq.com/blog/llm-api-cost-comparison
9. Gemini 1.5 Pro vs GPT-4o — Pricing, Benchmarks & Performance Compared, accessed March 17, 2026, https://anotherwrapper.com/tools/llm-pricing/gemini-15-pro/gpt-4o
10. What's the cheapest(good if free) but still useful LLM API in 2025? Also, which one is best for learning agentic AI? : r/AI_Agents - Reddit, accessed March 17, 2026, https://www.reddit.com/r/AI_Agents/comments/1m1ag00/whats_the_cheapestgood_if_free_but_still_useful/
11. The 11 best open-source LLMs for 2025 - n8n Blog, accessed March 17, 2026, https://blog.n8n.io/open-source-llm/
12. Skills - OpenClaw Docs, accessed March 17, 2026, https://docs.openclaw.ai/tools/skills
13. How to use Notion in openclaw? - Friends of the Crustacean - Answer Overflow, accessed March 17, 2026, https://www.answeroverflow.com/m/1472619432533688431
14. Structured Output Comparison across popular LLM providers — OpenAI, Gemini, Anthropic, Mistral and AWS Bedrock | by Rost Glukhov | Medium, accessed March 17, 2026,

https://medium.com/@rosgluk/structured-output-comparison-across-popular-llm-providers-openai-gemini-anthropic-mistral-and-1a5d42fa612a
15. Which LLM is best for JSON output while also being fast? : r/LLMDevs - Reddit, accessed March 17, 2026, https://www.reddit.com/r/LLMDevs/comments/1qvd1xz/which_llm_is_best_for_json_output_while_also/
16. Ultimate Guide - The Best Small LLMs For Personal Projects In 2026 - SiliconFlow, accessed March 17, 2026, https://www.siliconflow.com/articles/en/best-small-LLMs-for-personal-projects
17. Proton Mail and Calendar spring/summer roadmap, accessed March 17, 2026, https://proton.me/blog/mail-calendar-roadmap-spring-summer-2025
18. Proton Mail and Proton Calendar winter product roadmap, accessed March 17, 2026, https://proton.me/blog/mail-calendar-product-roadmap-winter
19. Calendar API - The Voice of the Proton Community - UserVoice, accessed March 17, 2026, https://protonmail.uservoice.com/forums/284483-proton-mail-calendar/suggestions/42931767-calendar-api
20. Tutorial — caldav 3.0.1 documentation, accessed March 17, 2026, https://caldav.readthedocs.io/stable/tutorial.html
21. Calendaring Extensions to WebDAV (CalDAV), accessed March 17, 2026, http://www.webdav.org/specs/rfc4791.html
22. Caldav-Tasks-API - Read the Docs, accessed March 17, 2026, https://caldavtasksapi.readthedocs.io/en/stable/
23. Documentation – Caldavsynchronizer, accessed March 17, 2026, https://caldavsynchronizer.org/help/documentation/
24. CalDAV: Support for categories · Issue #635 · tasks/tasks - GitHub, accessed March 17, 2026, https://github.com/tasks/tasks/issues/635
25. Getting started with Notion Calendar, accessed March 17, 2026, https://www.notion.com/help/guides/getting-started-with-notion-calendar
26. Working with databases - Notion Docs, accessed March 17, 2026, https://developers.notion.com/guides/data-apis/working-with-databases
27. Working with databases - Notion Docs, accessed March 17, 2026, https://developers.notion.com/docs/working-with-databases
28. Database properties – Notion Help Center, accessed March 17, 2026, https://www.notion.com/help/database-properties
29. Notion Database Properties Explained: A Complete Guide (2025) - NotionApps, accessed March 17, 2026, https://www.notionapps.com/blog/notion-database-properties-explained
30. Database - Notion Docs, accessed March 17, 2026, https://developers.notion.com/reference/database
31. Create a database - Notion Docs, accessed March 17, 2026, https://developers.notion.com/reference/create-a-database
32. Create a database – Notion Help Center, accessed March 17, 2026, https://www.notion.com/help/create-a-database
33. Filter database entries - Notion Docs, accessed March 17, 2026,

https://developers.notion.com/reference/post-database-query-filter
34. OpenClaw Notion Integration Guide | MI - 超智諮詢, accessed March 17, 2026, https://www.meta-intelligence.tech/en/insight-openclaw-notion.html
35. OpenClaw Notion Integration (The Fix That Actually Works) - YouTube, accessed March 17, 2026, https://www.youtube.com/watch?v=GC5Jn0L2eQA
36. Creating Skills - OpenClaw Docs, accessed March 17, 2026, https://docs.openclaw.ai/tools/creating-skills
37. Building Openclaw from Scratch — Part 3 (The Meta Skill), accessed March 17, 2026, https://systemdesigner.medium.com/building-openclaw-from-scratch-part-3-the-meta-skill-15a50fcb4384
38. 16 Best OpenClaw Skills for Your AI Agents - Firecrawl, accessed March 17, 2026, https://www.firecrawl.dev/blog/openclaw-skills
39. Building Custom OpenClaw Skills: A Hands-On Tutorial - DataCamp, accessed March 17, 2026, https://www.datacamp.com/pt/tutorial/building-open-claw-skills
40. Build Your Own AI Google Calendar Assistant with Agent Development Kit - Medium, accessed March 17, 2026, https://medium.com/google-cloud/build-your-own-ai-google-calendar-assistant-with-agent-development-kit-29f917be9e07
41. How to build a calendar concierge agent - Retool, accessed March 17, 2026, https://retool.com/blog/how-to-build-a-calendar-concierge-agent
42. 7 OpenClaw Agent Triggers You Must Know | by Tahir | Mar, 2026, accessed March 17, 2026, https://medium.com/@tahirbalarabe2/7-openclaw-agent-triggers-you-must-know-adfb212121b8
43. Cron vs Heartbeat - OpenClaw, accessed March 17, 2026, https://docs.openclaw.ai/automation/cron-vs-heartbeat
44. [OpenClaw] Cron jobs & background tasks execute but fail to send Telegram messages (silent failures) : r/AI_Agents - Reddit, accessed March 17, 2026, https://www.reddit.com/r/AI_Agents/comments/1qv8hl0/openclaw_cron_jobs_background_tasks_execute_but/
45. OpenClaw & Notion: 11 Things To Set Up In Week One - YouTube, accessed March 17, 2026, https://www.youtube.com/watch?v=YhhjndPa9aY&vl=de
46. Heartbeat - OpenClaw, accessed March 17, 2026, https://docs.openclaw.ai/gateway/heartbeat
47. How AI Gets Long-Term Memory (Ephemeral vs. Persistent Context) | Uplatz - YouTube, accessed March 17, 2026, https://www.youtube.com/watch?v=Ez37M1TGXBs
48. Ephemeral Context | Graphlit Agent Memory Glossary, accessed March 17, 2026, https://www.graphlit.com/glossary/ephemeral-context
49. Practical Memory Architecture For LLMs & What Works vs the Myth of “True Memory” : r/AI_Agents - Reddit, accessed March 17, 2026, https://www.reddit.com/r/AI_Agents/comments/1r92kfw/practical_memory_architecture_for_llms_what_works/
50. Ephemeral execution environments for AI agents in 2026 | Blog - Northflank,

accessed March 17, 2026, https://northflank.com/blog/ephemeral-execution-environments-ai-agents
51. OpenClaw Memory Masterclass: The complete guide to agent memory that survives, accessed March 17, 2026, https://velvetshark.com/openclaw-memory-masterclass
52. 8 Ways OpenClaw Reduces Context Loss in Long-Running Agents, accessed March 17, 2026, https://www.reddit.com/r/AI_Agents/comments/1quy0b9/8_ways_openclaw_reduces_context_loss_in/
53. Stop OpenClaw From Forgetting – The 3 Memory Layers Explained!, accessed March 17, 2026, https://www.youtube.com/watch?v=f8LJBh1AtKg
54. A Practical Guide to Securely Setting Up OpenClaw. I Replaced 6+ Apps with One "Digital Twin" on WhatsApp. | Medium, accessed March 17, 2026, https://medium.com/@srechakra/sda-f079871369ae
55. We Built Persistent Memory for OpenClaw (FKA Moltbot, ClawdBot) AI Agents - Mem0, accessed March 17, 2026, https://mem0.ai/blog/mem0-memory-for-openclaw
56. Built persistent memory for OpenClaw agents - no more context dumping : r/ClaudeCode, accessed March 17, 2026, https://www.reddit.com/r/ClaudeCode/comments/1r2qkx8/built_persistent_memory_for_openclaw_agents_no/
57. Persistent memory with high precision for long term and multi agent projects in LLM - Reddit, accessed March 17, 2026, https://www.reddit.com/r/claude/comments/1r4lb0r/persistent_memory_with_high_precision_for_long/
58. LLM Model Comparison: ChatGPT vs Claude vs Gemini vs Perplexity - Tacmind, accessed March 17, 2026, https://www.tacmind.com/blog/llm-model-comparison
59. Prompt caching - Claude API Docs, accessed March 17, 2026, https://platform.claude.com/docs/en/build-with-claude/prompt-caching
60. Prompt caching | OpenAI API, accessed March 17, 2026, https://developers.openai.com/api/docs/guides/prompt-caching/
61. How can I reduce API costs with repeated prompts? - OpenAI Developer Community, accessed March 17, 2026, https://community.openai.com/t/how-can-i-reduce-api-costs-with-repeated-prompts/1252602
62. Prompt Caching in the API - OpenAI, accessed March 17, 2026, https://openai.com/index/api-prompt-caching/
63. Prompt caching for faster model inference - Amazon Bedrock - AWS Documentation, accessed March 17, 2026, https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html
64. A quick guide on prompt caching with OpenAI, Anthropic, and Google - Dan's Working Notes, accessed March 17, 2026, https://promptthub.substack.com/p/a-quick-guide-on-prompt-caching-with
65. Prompt Caching and Token Costs in 2025: How to Spend Less Without Losing Quality, accessed March 17, 2026,

https://promptbuilder.cc/blog/prompt-caching-token-economics-2025
66. Tool Calling with Local LLMs: A Practical Evaluation - Docker, accessed March 17, 2026, https://www.docker.com/blog/local-llm-tool-calling-a-practical-evaluation/
67. Query a data source - Notion Docs, accessed March 17, 2026, https://developers.notion.com/reference/query-a-data-source
68. Post database query - Notion Developers, accessed March 17, 2026, https://developers.notion.com/reference/post-database-query